VSP TECHVERSE
An Institute of AI & Robotics

Decision Tree & Random Forest for AI & Robotics

Teaching a robot to decide by asking simple yes/no questions



VSP TECHVERSE
An Institute of AI & Robotics

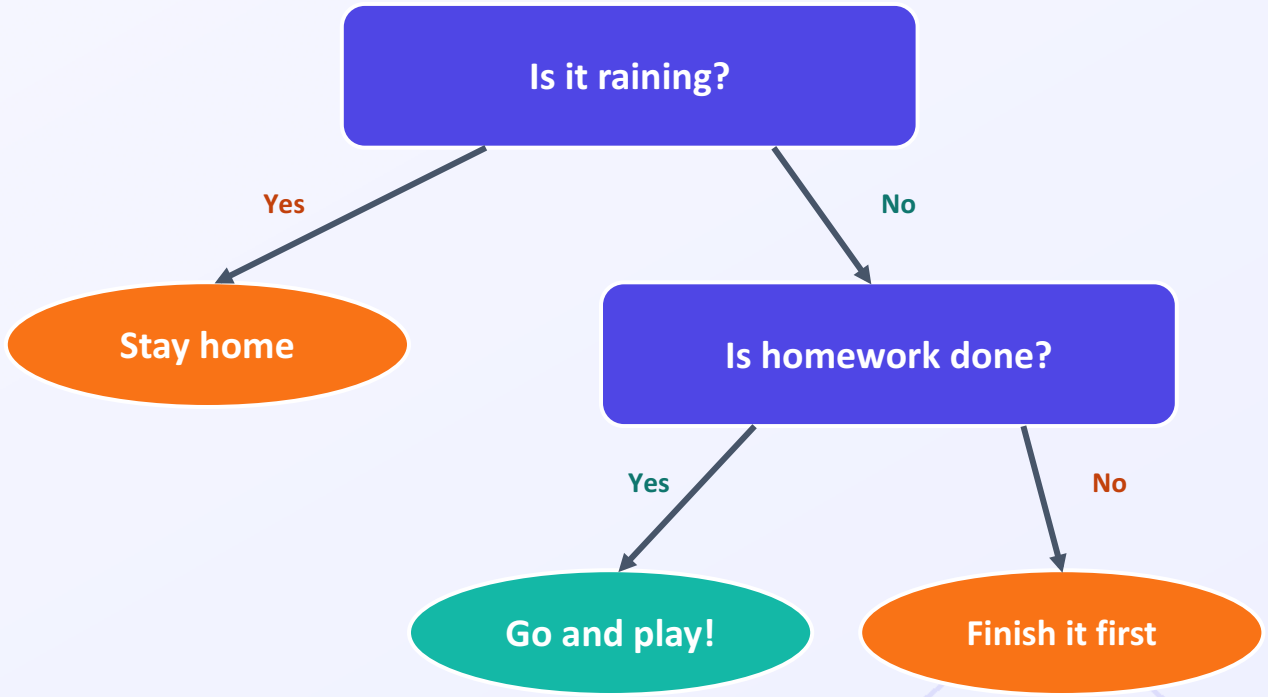
LET'S START SIMPLE

You Already Think Like a Decision Tree

Every day you make decisions by asking yourself a few simple questions, one after another.

- Each question has a yes or no answer
- Each answer leads to the next question
- You stop when you reach a final decision
- A decision tree is exactly this, written as a chart

SHOULD I GO OUT TO PLAY CRICKET?



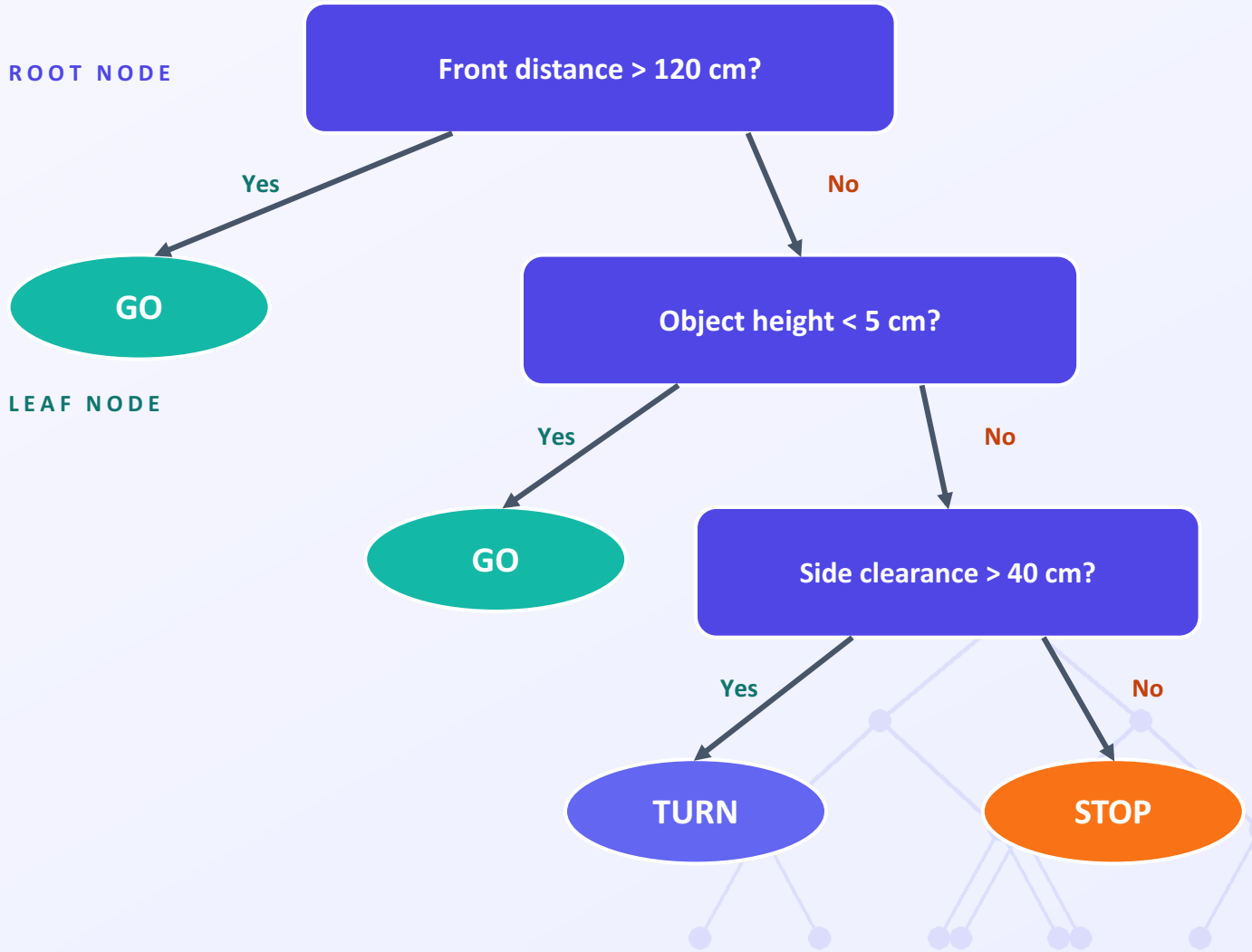
Questions = decision nodes · Final answers = leaf nodes

KEY IDEA A machine learning model can learn these questions by itself from data.

What is a Decision Tree?

A decision tree is a machine learning model that predicts an answer by asking a series of yes/no questions about the data, like a flowchart.

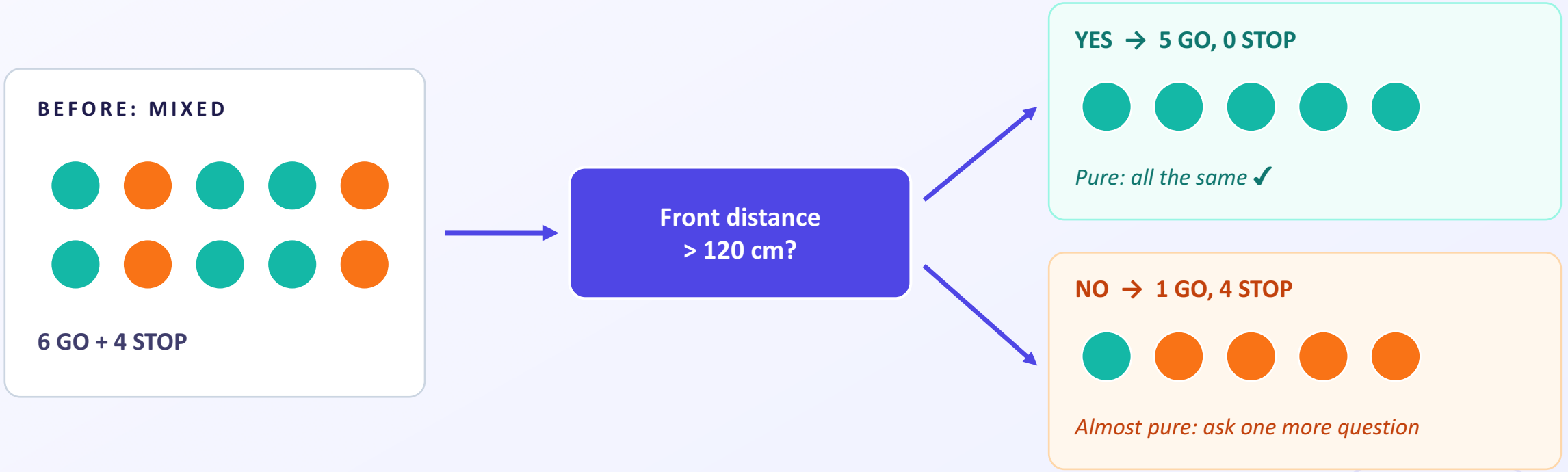
- **Root node:** the first question at the top
- **Decision node:** any question in the middle
- **Branch:** the yes / no path
- **Leaf node:** the final answer
- **Depth:** how many levels of questions



ROBOTICS EXAMPLE Our robot must decide: GO, TURN or STOP?

How Does the Tree Choose a Question?

The tree tries many possible questions and picks the one that separates the data best: each side should contain mostly one kind of answer.



Gini impurity is the score scikit-learn uses to measure how mixed a group is: **0 = pure (all one class)** · higher = more mixed. Next we learn how to calculate it.

Gini Impurity: How Mixed is a Group?

Gini impurity gives one group of data a score. It tells us the chance of a wrong guess if we picked a random row and labelled it randomly by the group's mix.

FORMULA (p_i = share of class i)

$$\text{Gini} = 1 - (p_1^2 + p_2^2 + \dots)$$

STEP BY STEP: 6 GO + 4 STOP

Step 1: p(GO) = 6/10 = 0.6, p(STOP) = 4/10 = 0.4

Step 2: square them → 0.36 and 0.16

Step 3: add them → 0.36 + 0.16 = 0.52

Step 4: Gini = 1 − 0.52 = 0.48

Lower Gini = purer group = better.

THREE EXAMPLES



5 GO, 0 STOP

$1 - (1^2 + 0^2) = 0$ **Pure**



1 GO, 4 STOP

$1 - (0.2^2 + 0.8^2) = 0.32$ **Fairly pure**



3 GO, 3 STOP

$1 - (0.5^2 + 0.5^2) = 0.5$ **Most mixed (2 classes)**

Gini Index: Scoring a Whole Question

A question splits one group into two. The Gini index of the question is the weighted average of the two groups' Gini impurity. The tree picks the question with the lowest Gini index.

$$\text{Gini index} = (\text{rows left} / \text{total}) \times \text{Gini_left} + (\text{rows right} / \text{total}) \times \text{Gini_right}$$

QUESTION A: Front distance > 120 cm?

```
YES: 5 GO, 0 STOP  →  Gini 0
NO:  1 GO, 4 STOP  →  Gini 0.32
```

$$\text{Gini index} = 5/10 \times 0 + 5/10 \times 0.32 = 0.16$$

WINNER ✓ lowest score

QUESTION B: Battery > 50%?

```
YES: 3 GO, 2 STOP  →  Gini 0.48
NO:  3 GO, 2 STOP  →  Gini 0.48
```

$$\text{Gini index} = 5/10 \times 0.48 + 5/10 \times 0.48 = 0.48$$

Not useful ✗ still mixed

Note: many books use "Gini index" and "Gini impurity" to mean the same thing. Here, impurity scores one group and index scores a whole split.

Entropy and Information Gain

Entropy is another way to measure how mixed a group is. It comes from information theory and uses log base 2.

ENTROPY

$$\text{Entropy} = -\Sigma\, p_i \times \log_2(p_i)$$

INFORMATION GAIN (E = entropy, after = weighted)

$$\text{Gain} = E(\text{before}) - E(\text{after})$$

- Entropy 0 → pure group
- Entropy 1 → 50/50 mix (two classes)
- The tree picks the question with the highest information gain

`Use it in code with: criterion="entropy"`

STEP BY STEP: QUESTION A (front distance)

Before (6 GO, 4 STOP)

$$-(0.6 \times \log_2 0.6 + 0.4 \times \log_2 0.4) = 0.97$$

YES group (5 GO, 0 STOP)

$$\text{pure} \rightarrow \text{entropy} = 0$$

NO group (1 GO, 4 STOP)

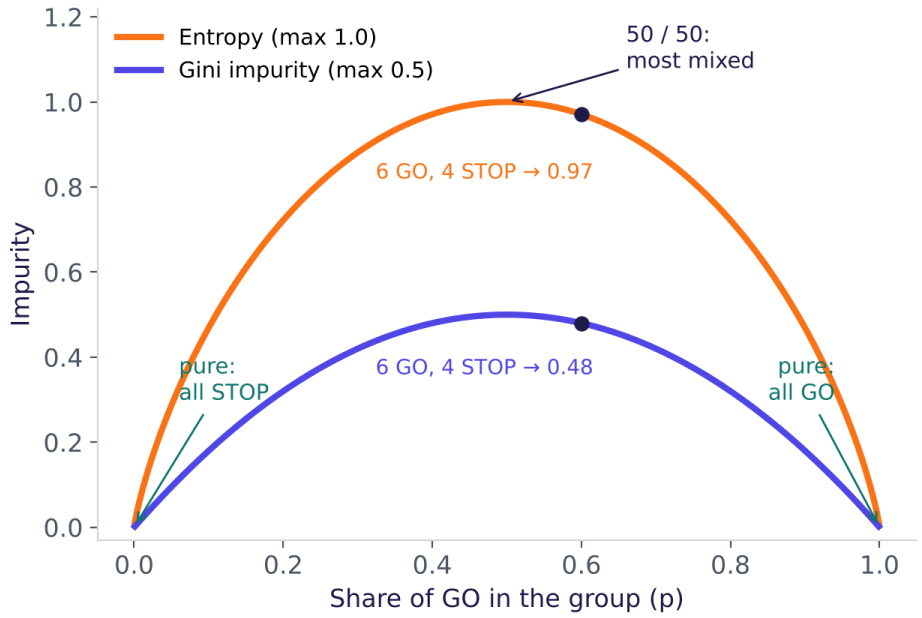
$$-(0.2 \times \log_2 0.2 + 0.8 \times \log_2 0.8) = 0.72$$

Weighted after = 5/10 × 0 + 5/10 × 0.72 = 0.36

Information gain = 0.97 − 0.36 = 0.61

Question B (battery) has gain 0: it teaches the tree nothing.

Gini vs Entropy: Which One to Use?



	Gini impurity	Entropy
Range (2 classes)	0 to 0.5	0 to 1
Uses	squares	$\log_2$
Speed	a bit faster	a bit slower
In scikit-learn	default	criterion="entropy"

criterion.py

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

df = pd.read_csv("robot_navigation_data.csv")
X = df[["front_distance_cm", "side_clearance_cm",
        "object_height_cm", "battery_pct"]]
y = df["action"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

for c in ["gini", "entropy"]:
    model = DecisionTreeClassifier(criterion=c, max_depth=4,
                                   random_state=42)
    model.fit(X_train, y_train)
    print(c, "accuracy:", round(model.score(X_test, y_test), 2))
```

OUTPUT

```
gini accuracy: 0.88
entropy accuracy: 0.9
```

Both are 0 for pure groups and highest at a 50/50 mix, so they usually pick the same questions and give similar results.

Our Robot Datasets

We use two small CSV files generated with NumPy. One teaches classification (choose a category), the other regression (predict a number).

CLASSIFICATION
`robot_navigation_data.csv`

300 rows

INPUTS (X)

- **`front_distance_cm`** distance to the obstacle ahead
- **`side_clearance_cm`** free space at the side
- **`object_height_cm`** height of the obstacle
- **`battery_pct`** battery level

ANSWER (y) `action: go / turn / stop`

What should the robot do next?

REGRESSION
`robot_runtime_data.csv`

300 rows

INPUTS (X)

- **`payload_kg`** weight the robot carries
- **`speed_mps`** driving speed
- **`ambient_temp_c`** outside temperature

ANSWER (y) `runtime_min (minutes)`

How long will the battery last?

6 Steps to Build Any ML Model

1. Load

Read the CSV file with pandas

```
pd.read_csv()
```

2. X and y

Pick the inputs and the answer

```
df[[...]], df["..."]
```

3. Split

Keep 20% of rows for testing

```
train_test_split()
```

4. Train

Create the model and let it learn

```
model.fit()
```

5. Predict

Ask the model about the test rows

```
model.predict()
```

6. Check

Compare predictions with real answers

```
metrics, plots
```

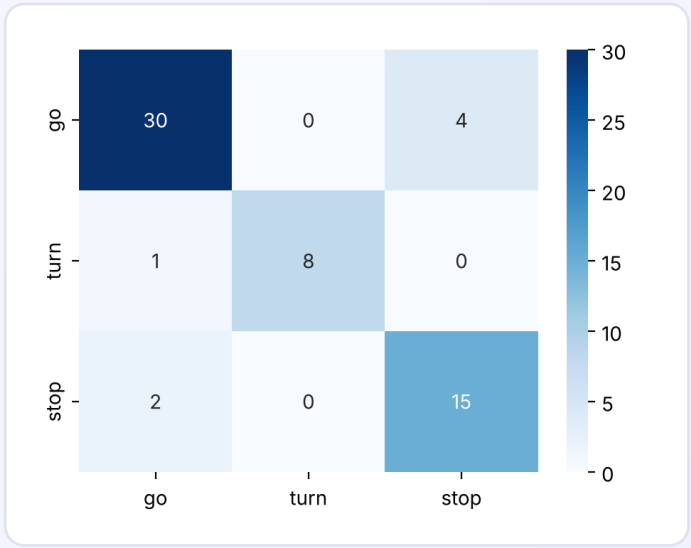
Every code example in this class follows these same 6 steps, marked as comments in the code.

Code: Decision Tree Classifier

decision_tree.py

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn import metrics
# 1. Load the robot data
df = pd.read_csv("robot_navigation_data.csv")
# 2. Inputs (X) and the answer to learn (y)
X = df[["front_distance_cm", "side_clearance_cm",
        "object_height_cm", "battery_pct"]]
y = df["action"]
# 3. Keep 20% of the rows aside for testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
# 4. Build the tree and train it
model = DecisionTreeClassifier(max_depth=4, random_state=42)
model.fit(X_train, y_train)
# 5. Predict on the test data and check
y_pred = model.predict(X_test)
print("Accuracy:", round(metrics.accuracy_score(y_test, y_pred), 2))
# 6. Confusion matrix: rows = actual, columns = predicted
labels = ["go", "turn", "stop"]
cm = metrics.confusion_matrix(y_test, y_pred, labels=labels)
sns.heatmap(cm, annot=True, cmap="Blues",
            xticklabels=labels, yticklabels=labels)
plt.show()
```

CONFUSION MATRIX (TEST DATA)



OUTPUT

```
Accuracy: 0.88
```

Accuracy 0.88 = 88% of test readings correct. Diagonal boxes = correct answers, other boxes = mistakes.

Code: Draw the Tree

plot_tree.py

```
from sklearn.tree import plot_tree

# a small tree (3 levels) is easy to read
small = DecisionTreeClassifier(max_depth=3, random_state=42)
small.fit(X_train, y_train)

plt.figure(figsize=(14, 6))
plot_tree(small, feature_names=X.columns,
          class_names=small.classes_, filled=True)
plt.show()
```

Add this after step 3 of the previous code.

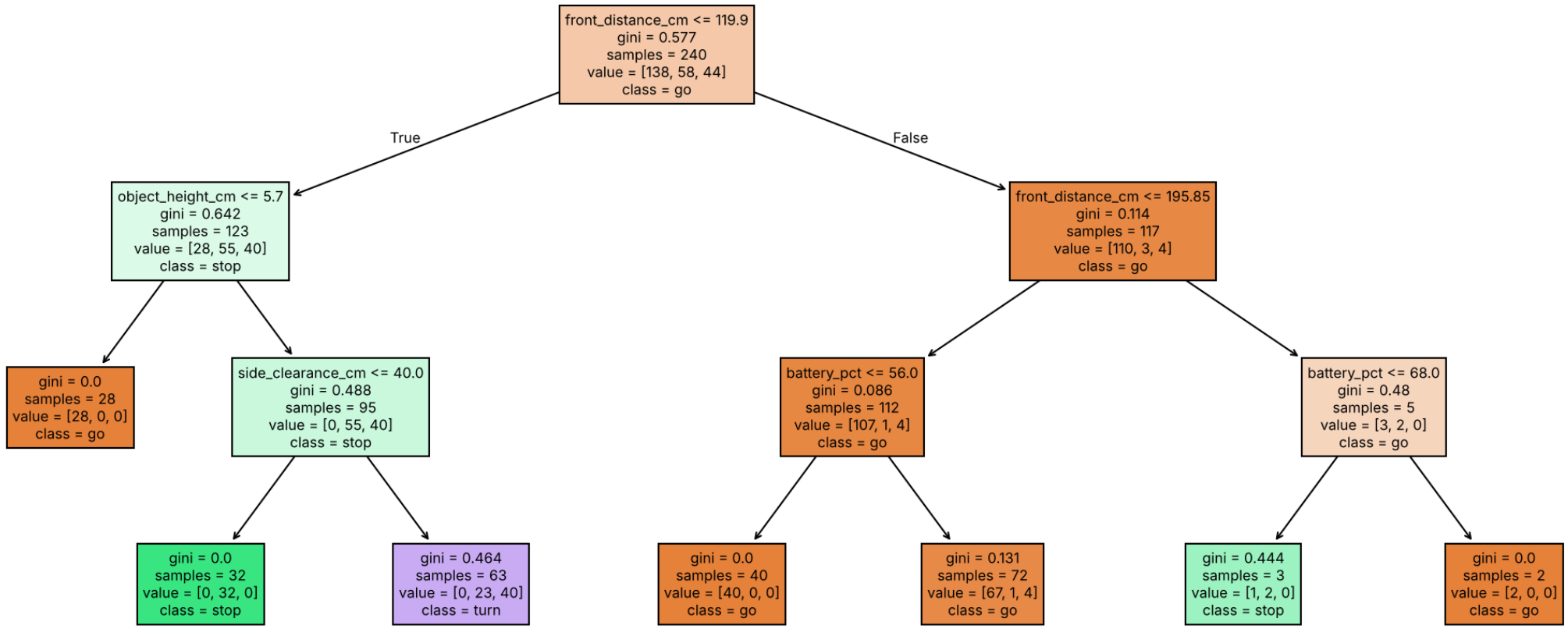
WHY IT MATTERS We can read the robot's rules and check they make sense, a big plus for safety.

HOW TO READ ONE BOX

```
front_distance_cm <= 119.9
gini = 0.577
samples = 240
value = [138, 58, 44]
class = go
```

- **Line 1:** the question (True → left, False → right)
- **gini:** how mixed the box is (0 = pure)
- **samples:** rows that reached this box
- **value:** how many go, stop, turn
- **class:** the answer if we stopped here

The Tree Our Robot Learned

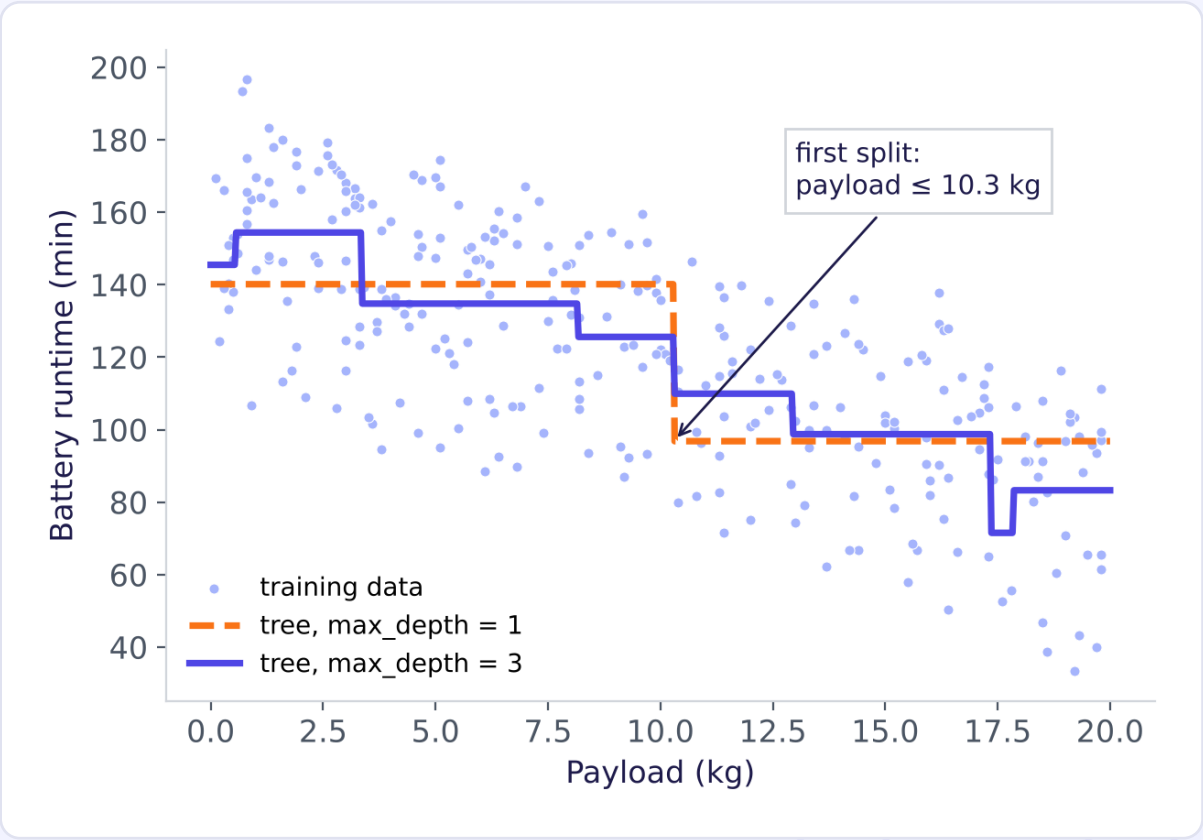


It learned our rules by itself: first front distance, then object height, then side clearance.

Decision Tree for Regression

A tree can also predict a number. It asks questions in the same way, and each leaf gives the average value of the training rows that ended there.

- The prediction looks like a staircase
- More levels (depth) = more steps
- Each step = one leaf = one average value



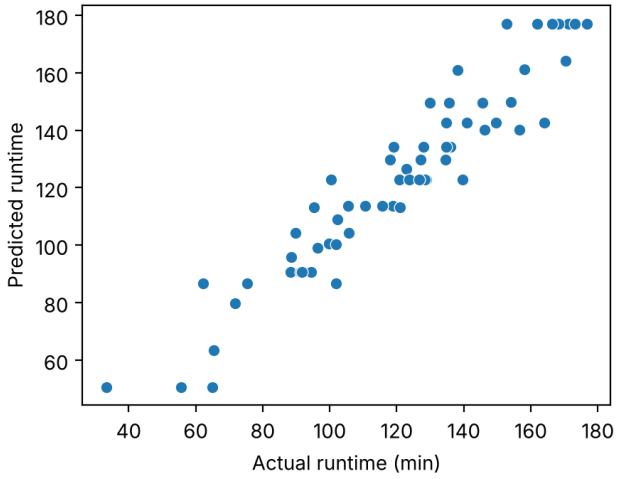
ROBOTICS EXAMPLE First question: is the payload ≤ 10.3 kg? Light loads → about 140 min of runtime, heavy loads → about 97 min.

Code: Decision Tree Regressor

tree_regressor.py

```python
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn import metrics
# 1. Load the robot data
df = pd.read_csv("robot_runtime_data.csv")
# 2. Inputs (X) and the answer to learn (y)
X = df[["payload_kg", "speed_mps", "ambient_temp_c"]]
y = df["runtime_min"]
# 3. Keep 20% of the rows aside for testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
# 4. Build the tree and train it
model = DecisionTreeRegressor(max_depth=5, random_state=42)
model.fit(X_train, y_train)
# 5. Predict and check the error
y_pred = model.predict(X_test)
print("MAE:", round(metrics.mean_absolute_error(y_test, y_pred), 1))
print("R2 :", round(metrics.r2_score(y_test, y_pred), 2))
# 6. Plot actual vs predicted
sns.scatterplot(x=y_test, y=y_pred)
plt.xlabel("Actual runtime (min)"); plt.ylabel("Predicted runtime")
plt.show()
```

ACTUAL vs PREDICTED



OUTPUT

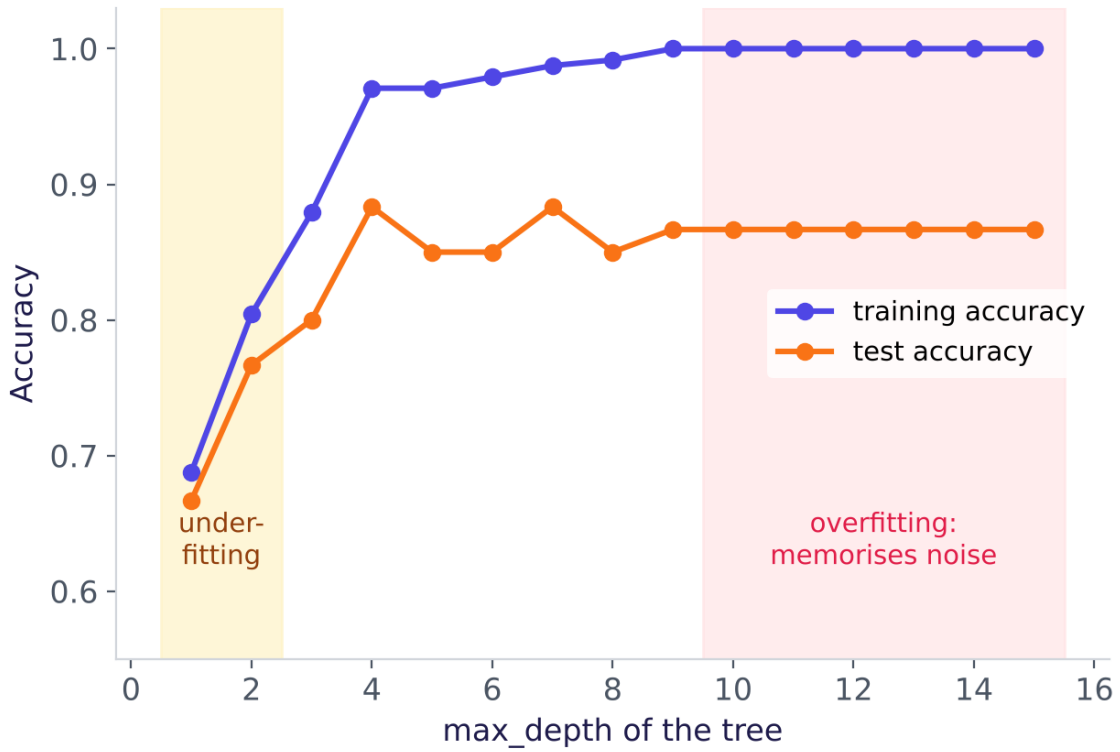
```
MAE: 8.2
R2 : 0.89
```

MAE 8.2 → predictions are about 8 minutes off on average. R^2 0.89 → a good model (1.0 would be perfect).

Too Many Questions = Overfitting

Like a student who memorises answers: 100% on practice questions, but poor marks on new exam questions.

- A very deep tree memorises the training data, even its mistakes
- Training accuracy reaches 100%, test accuracy does not
- Fix: limit the depth, e.g. max_depth=4
- Too few levels is also bad (underfitting)



ROBOTICS EXAMPLE Our best test result is at depth 4 (88%).

Decision Tree: Good & Bad

ADVANTAGES

- Easy to understand: just yes/no questions
- We can draw it and read its rules
- Works for categories and numbers
- No need to scale the data

DISADVANTAGES

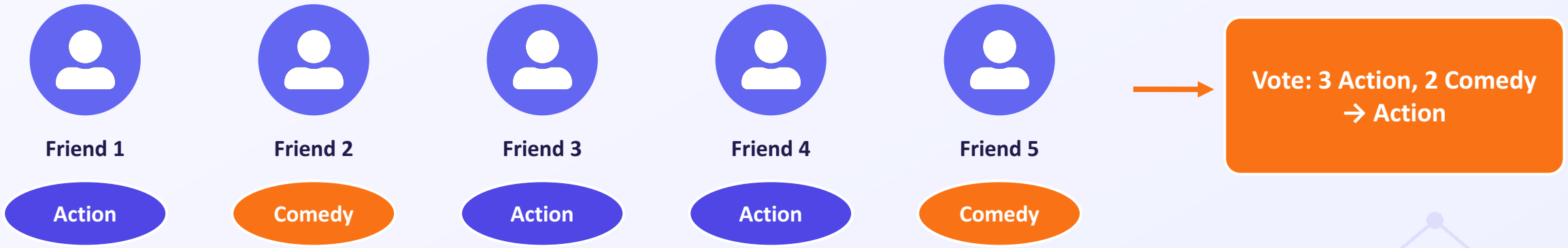
- Overfits easily if it grows too deep
- A small change in data can change the whole tree
- Regression predictions look like steps
- Often less accurate than a random forest

BEST FIT IN ROBOTICS Simple robot rules that people must be able to check, like safety stops and fault diagnosis.

What is a Random Forest?

A random forest is a team of many decision trees. Each tree gives its own answer, and the forest picks the answer most trees agree on.

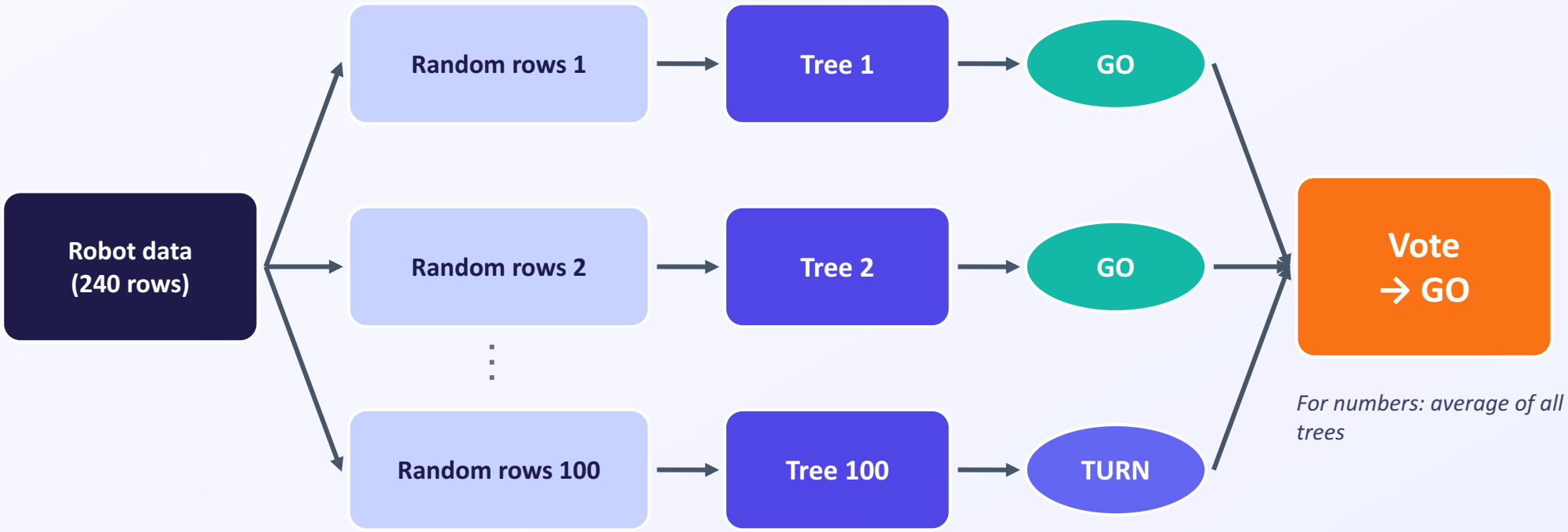
LIKE ASKING FRIENDS: "WHICH MOVIE SHOULD I WATCH?"



One friend can be wrong; the group is usually right.

IN OUR ROBOT Robot: 100 trees each say GO, TURN or STOP; the forest chooses the most common answer.

How is a Random Forest Built?



For numbers: average of all trees

Different trees make different mistakes, so the mistakes cancel out in the vote.

1 Random rows
Each tree learns from a different random sample of the data.

2 Random questions
At each step a tree only looks at a few random sensors, so the trees are all different.

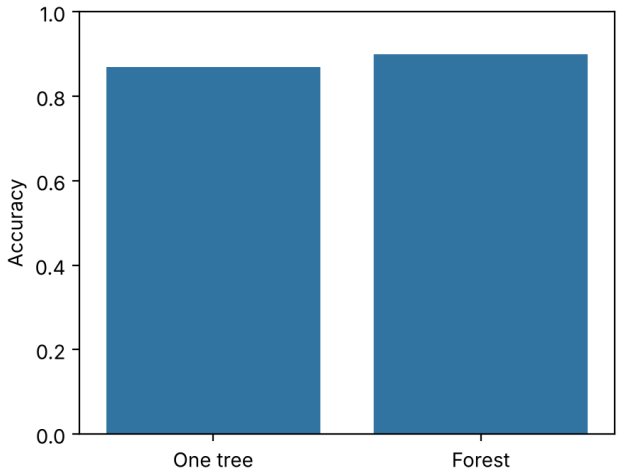
3 Combine
Categories → majority vote. Numbers → average.

Code: One Tree vs a Random Forest

random_forest.py

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn import metrics
# 1. Load the robot data
df = pd.read_csv("robot_navigation_data.csv")
# 2. Inputs (X) and the answer to learn (y)
X = df[["front_distance_cm", "side_clearance_cm",
        "object_height_cm", "battery_pct"]]
y = df["action"]
# 3. Keep 20% of the rows aside for testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
# 4. Train ONE tree and a FOREST of 100 trees
tree = DecisionTreeClassifier(random_state=42)
tree.fit(X_train, y_train)
forest = RandomForestClassifier(n_estimators=100, random_state=42)
forest.fit(X_train, y_train)
# 5. Compare their accuracy on the test data
tree_acc = round(tree.score(X_test, y_test), 2)
forest_acc = round(forest.score(X_test, y_test), 2)
print("One tree :", tree_acc)
print("Forest   :", forest_acc)
sns.barplot(x=["One tree", "Forest"], y=[tree_acc, forest_acc])
plt.ylim(0, 1); plt.ylabel("Accuracy")
plt.show()
```

ACCURACY: ONE TREE vs FOREST



OUTPUT

```
One tree : 0.87
Forest   : 0.9
```

Same data, same test: one tree scores 0.87, the forest scores 0.90.

Which Sensor Matters Most?

importance.py

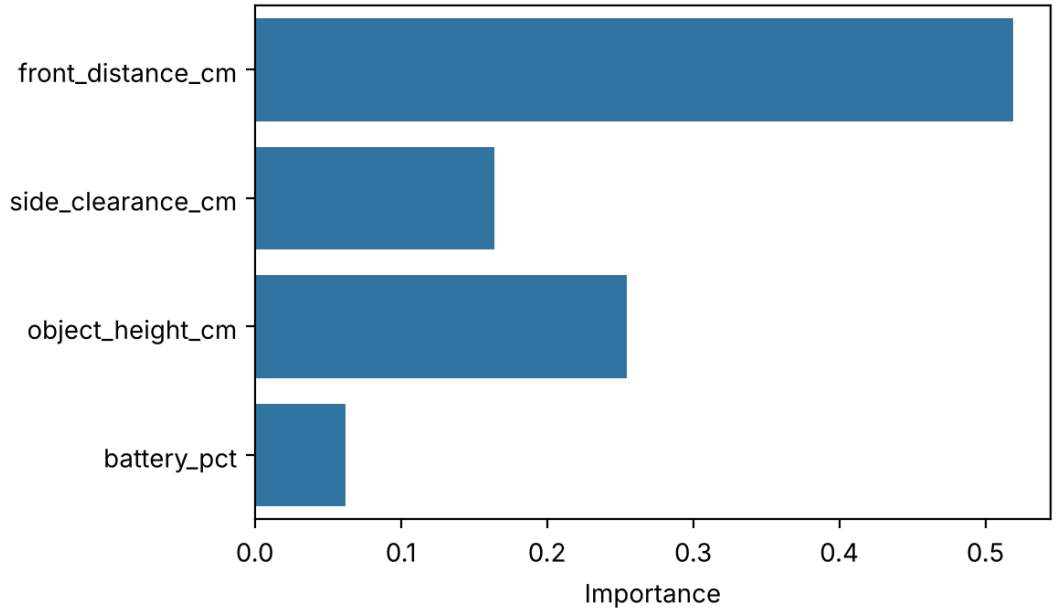
```
# how much each sensor helped the forest decide
importance = pd.Series(forest.feature_importances_,
                       index=X.columns)
print(importance.round(2))

sns.barplot(x=importance.values, y=importance.index)
plt.xlabel("Importance"); plt.ylabel("")
plt.show()
```

OUTPUT

```
front_distance_cm    0.52
side_clearance_cm    0.16
object_height_cm     0.25
battery_pct          0.06
dtype: float64
```

Add this after the forest code on the previous slide.



Front distance is the most useful sensor. **Battery level** hardly matters for choosing the action, and the forest found this by itself.

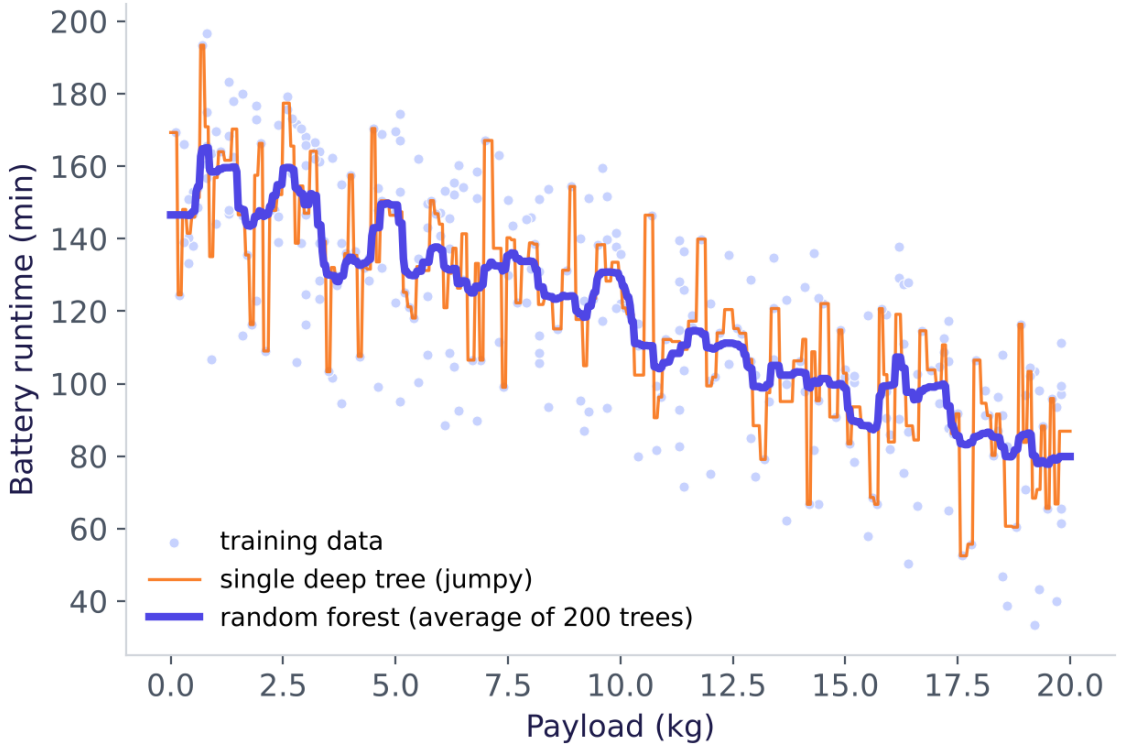
Random Forest for Regression

For numbers, every tree makes a prediction and the forest gives the average of all of them.

PREDICTION

Forest answer = average of all trees

- One deep tree jumps up and down (orange line)
- The average of 200 trees is smooth (blue line)
- Smoother usually means better on new data



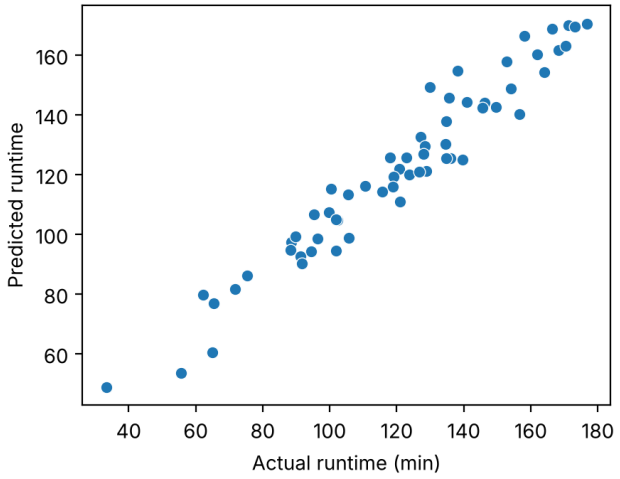
ROBOTICS EXAMPLE A steady battery-time estimate that does not jump when the load changes a little.

Code: Random Forest Regressor

forest_regressor.py

```python
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn import metrics
# 1. Load the robot data
df = pd.read_csv("robot_runtime_data.csv")
# 2. Inputs (X) and the answer to learn (y)
X = df[["payload_kg", "speed_mps", "ambient_temp_c"]]
y = df["runtime_min"]
# 3. Keep 20% of the rows aside for testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
# 4. Build a forest of 200 trees and train it
model = RandomForestRegressor(n_estimators=200, random_state=42)
model.fit(X_train, y_train)
# 5. Predict and check the error
y_pred = model.predict(X_test)
print("MAE:", round(metrics.mean_absolute_error(y_test, y_pred), 1))
print("R2 :", round(metrics.r2_score(y_test, y_pred), 2))
# 6. Plot actual vs predicted
sns.scatterplot(x=y_test, y=y_pred)
plt.xlabel("Actual runtime (min)"); plt.ylabel("Predicted runtime")
plt.show()
```

ACTUAL vs PREDICTED



OUTPUT

```
MAE: 6.6
R2 : 0.94
```

Better than one tree: MAE 8.2 → 6.6 minutes, R^2 0.89 → 0.94.

Random Forest: Good & Bad

ADVANTAGES

- More accurate than a single tree
- Much less overfitting
- Works for categories and numbers
- Tells us which inputs matter most

DISADVANTAGES

- Slower, because it trains many trees
- Uses more memory
- Harder to explain: no single tree to read
- Cannot predict values far outside the training data

BEST FIT IN ROBOTICS Reliable predictions from many sensors: obstacle and terrain detection, battery and fault prediction.

Decision Tree vs Random Forest

	Decision Tree	Random Forest
What is it?	One tree of yes/no questions	A team of many trees
How it answers	Follows the questions to a leaf	Trees vote (or average)
Easy to understand?	Yes, we can draw it	Harder
Overfitting	Easily, if too deep	Much less
Speed	Fast	Slower
Robot navigation (accuracy)	0.88	0.90
Battery runtime (R^2)	0.89	0.94

Key Words to Remember

Root node
The first question at the top of the tree

Leaf node
A final answer at the bottom

Depth / max_depth
How many levels of questions the tree can ask

Gini impurity
How mixed one group is (0 = pure)

Gini index
Weighted Gini of a whole split (lower = better question)

Entropy
Another mixed-ness score using $\log_2$ (0 = pure, 1 = 50/50)

Information gain
Drop in entropy after a question (higher = better)

Overfitting
Memorising training data and failing on new data

Random forest
Many trees that vote or average their answers

Feature importance
How useful each input was to the model

VSP TECHVERSE
An Institute of AI & Robotics

Thank You

Keep learning, keep building



VSP TECHVERSE
An Institute of AI & Robotics